

SECTION D

Step 1:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view containing 'con', 'foodapp_db', 'Tables', 'Views', 'Stored Procedures', 'Functions', 'Instadone', 'stud', and 'sys'. The 'foodapp_db' schema is selected. The main editor window shows a SQL script with the following content:

```
392 COUNT(order_id) AS total_orders,  
393 SUM(total_amount) AS total_revenue,  
394 ROUND(AVG(total_amount), 2) AS average_order_value  
395 FROM orders  
396 WHERE restaurant_id = p_restaurant_id  
397 AND MONTH(order_date) = p_month  
398 GROUP BY restaurant_id, MONTH(order_date);  
399  
400 END IF;  
401  
402 END IF;  
403 END //  
404  
405 DELIMITER ;  
406 CALL restaurant_monthly_summary(1, 8);
```

The 'Result Grid' tab is active, showing a message: 'No orders found for restaurant ID 1 in month 8.' The 'Output' tab is also active, showing the 'Action Output' table with the following data:

#	Time	Action	Message	Duration / Fetch
61	17:31:58	SELECT * FROM order_audit LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
62	18:00:24	CREATE PROCEDURE restaurant_monthly_summary(IN p_restaurant_id INT, IN p_month INT) BEGIN ...	0 row(s) affected	0.015 sec
63	18:00:37	CALL restaurant_monthly_summary(1, 8)	1 row(s) returned	0.000 sec / 0.000 sec

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view containing 'con', 'foodapp_db', 'Tables', 'Views', 'Stored Procedures', 'Functions', 'Instadone', 'stud', and 'sys'. The 'foodapp_db' schema is selected. The main editor window shows a SQL script with the following content:

```
393 SUM(total_amount) AS total_revenue,  
394 ROUND(AVG(total_amount), 2) AS average_order_value  
395 FROM orders  
396 WHERE restaurant_id = p_restaurant_id  
397 AND MONTH(order_date) = p_month  
398 GROUP BY restaurant_id, MONTH(order_date);  
399  
400 END IF;  
401  
402 END IF;  
403 END //  
404  
405 DELIMITER ;  
406 CALL restaurant_monthly_summary(1, 8);  
407 CALL restaurant_monthly_summary(2, 9);
```

The 'Result Grid' tab is active, showing a message: 'No orders found for restaurant ID 2 in month 9.' The 'Output' tab is also active, showing the 'Action Output' table with the following data:

#	Time	Action	Message	Duration / Fetch
62	18:00:24	CREATE PROCEDURE restaurant_monthly_summary(IN p_restaurant_id INT, IN p_month INT) BEGIN ...	0 row(s) affected	0.015 sec
63	18:00:37	CALL restaurant_monthly_summary(1, 8)	1 row(s) returned	0.000 sec / 0.000 sec
64	18:00:48	CALL restaurant_monthly_summary(2, 9)	1 row(s) returned	0.000 sec / 0.000 sec

Step 2:

Q1.

Ans:

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a stored procedure that:

- Accepts a restaurant_id and a month number (1–12) as inputs and returns a summary showing total orders, total revenue, and average order value for that restaurant in that month.
- Handles the case where the restaurant has no orders in that month by returning a meaningful message rather than an empty result.
- Uses a SELECT with GROUP BY and aggregate functions to compute the summary values correctly.
- Runs successfully in MySQL Workbench — screenshot output for at least two different restaurant_id and month combinations.

Q2.

Ans:

```
DELIMITER //

CREATE PROCEDURE restaurant_monthly_summary(
    IN p_restaurant_id INT,
    IN p_month INT
)
BEGIN
    DECLARE order_count INT DEFAULT 0;

    -- Check whether the month number is valid
    IF p_month < 1 OR p_month > 12 THEN

        SELECT 'Invalid month. Please enter a month between 1 and 12.' AS message;

    ELSE

        -- Check whether orders exist
        SELECT COUNT(*)
        INTO order_count
        FROM orders
        WHERE restaurant_id = p_restaurant_id
        AND MONTH(order_date) = p_month;

        IF order_count = 0 THEN

            SELECT CONCAT(
                'No orders found for restaurant ID ',
                p_restaurant_id,
                ' in month ',
                p_month,
                ''
            ) AS message;

        ELSE

            -- Generate monthly summary
            SELECT
```

```

        restaurant_id,
        MONTH(order_date) AS month,
        COUNT(order_id) AS total_orders,
        SUM(total_amount) AS total_revenue,
        ROUND(AVG(total_amount), 2) AS average_order_value
    FROM orders
    WHERE restaurant_id = p_restaurant_id
        AND MONTH(order_date) = p_month
    GROUP BY restaurant_id, MONTH(order_date);

END IF;

END IF;
END //

DELIMITER ;
CALL restaurant_monthly_summary(1, 8);
CALL restaurant_monthly_summary(2, 9);
CALL restaurant_monthly_summary(3, 12);

```

Q3.

Ans:

I modified the procedure to validate the month number and check whether the restaurant has any orders before generating the summary.

I added a meaningful “No orders found” message instead of returning an empty result.

I also used COUNT(), SUM(), AVG(), and GROUP BY to calculate the monthly statistics correctly.

The AI’s original version needed improvement because it did not properly handle invalid months and cases with no orders.